

API Specification và OpenAPI

Nhóm 10

|

Nội dung

01

Giới thiệu OpenAPI và Swagger

Phân biệt specification và toolset, lịch sử phát triển

02

Vai trò của OpenAPI

Tại sao cần tài liệu hóa API, lợi ích thực tế

03

Cấu trúc file OpenAPI

Tổng quan cấu trúc YAML/JSON, các thành phần chính

04

Paths và Operations

Định nghĩa endpoints, HTTP methods, parameters

05

Definitions

Tái sử dụng data models

06

Swagger UI

Tạo tài liệu tự động

OpenAPI và Swagger



OpenAPI

Định nghĩa

Specification - định dạng chuẩn để mô tả RESTful APIs

Vai trò

Như bản thiết kế cho API - định nghĩa cấu trúc, endpoints, parameters, responses

Đặc điểm

Vendor-neutral, machine-readable, language-agnostic



Swagger

Định nghĩa

Toolset - bộ công cụ để làm việc với OpenAPI

Vai trò

Như công cụ xây dựng - giúp tạo, hiển thị, và tương tác với OpenAPI specs

Công cụ

Swagger UI, Swagger Editor, Swagger Codegen



Lịch sử & Quan hệ

Swagger Specification được SmartBear Software donate cho Linux Foundation năm 2015 và đổi tên thành **OpenAPI Specification**. Các công cụ Swagger vẫn giữ tên cũ và tiếp tục phát triển để hỗ trợ OpenAPI 3.0+

Vai trò của OpenAPI trong tài liệu hóa API



Chuẩn hóa & Nhất quán

- Định dạng vendor-neutral được công nhận toàn cầu
- Mọi developer đều có thể hiểu API dù không quen backend
- Thúc đẩy sự nhất quán trong việc tài liệu hóa API



Cải thiện Developer Experience

- Rõ ràng: endpoints, parameters, responses
- Interactive documentation - Test API ngay trong browser
- Giảm learning curve, làm quen nhanh hơn



Tự động hóa

- Tự động tạo client SDKs (Python, Java, JS...)
- Tự động tạo server stubs và boilerplate code

Cấu trúc file OpenAPI – Tổng quan

Các thành phần chính

- 1 Metadata**
Swagger version, info (title, version)
- 2 Base URL**
Host, basePath
- 3 Consumes, Produces**
Content-Type mà API hỗ trợ
- 4 Paths**
Endpoints & HTTP methods
- 5 Parameters**
Là path parameters, query string, header hoặc request body
- 6 Responses**
Gồm status code và response body
- 7 Definitions**
Định nghĩa những thành phần để tái sử dụng

Ví dụ: Cấu trúc file YAML

```
swagger: "2.0"
info:
  title: Book Management API
  version: "1.0"
host: localhost:5000
basePath: /
consumes:
  - application/json
produces:
  - application/json
paths:
  /api/v1/books:
    post:
      responses:
        "201":
          description: Created
          summary: Add new book
          operationId: post_book_list
          parameters:
            - name: payload
              required: true
              in: body
              schema:
                $ref: "#/definitions/Book"
      tags:
        - api/v1/books
```

Paths, Responses, Parameters

Paths (Endpoints)

Định nghĩa

Mỗi path là một endpoint API, bắt đầu bằng /

Path Templating

Sử dụng {parameter}

[/users/{userId}](#)

HTTP Methods cho các endpoint

GET

Read

POST

Create

PUT

Update

PATCH

Partial

DELETE

Remove

Ví dụ: `/api/v1/books` và `/api/v1/books/{id}`

```
paths:
  /api/v1/books:
    post:
      responses:
        "201":
          description: Created
      summary: Add new book
      operationId: post_book_list
      parameters:
        - name: payload
          required: true
          in: body
          schema:
            $ref: "#/definitions/Book"
      tags:
        - api/v1/books
  /api/v1/books/{book_id}:
    parameters:
      - name: book_id
        in: path
        required: true
        type: integer
    get:
      responses:
        "200":
          description: Success
      summary: Get a specific book
      operationId: get_book_item
      tags:
        - api/v1/books
```

Definitions

Definitions

Nơi định nghĩa các thành phần **tái sử dụng** trong toàn bộ specification

Sử dụng bằng \$ref: `$ref: "#/definitions/Book"`

Ví dụ: Sử dụng Book

```
paths:
  /api/v1/books:
    post:
      responses:
        "201":
          description: Created
      summary: Add new book
      operationId: post_book_list
      parameters:
        - name: payload
          required: true
          in: body
          schema:
            $ref: "#/definitions/Book"
      tags:
        - api/v1/books
```

Ví dụ: Definition Book

```
definitions:
  Book:
    required:
      - title
    properties:
      id:
        type: integer
      title:
        type: string
      author:
        type: string
        default: Unknown
      year:
        type: integer
        default: 2026
    type: object
```

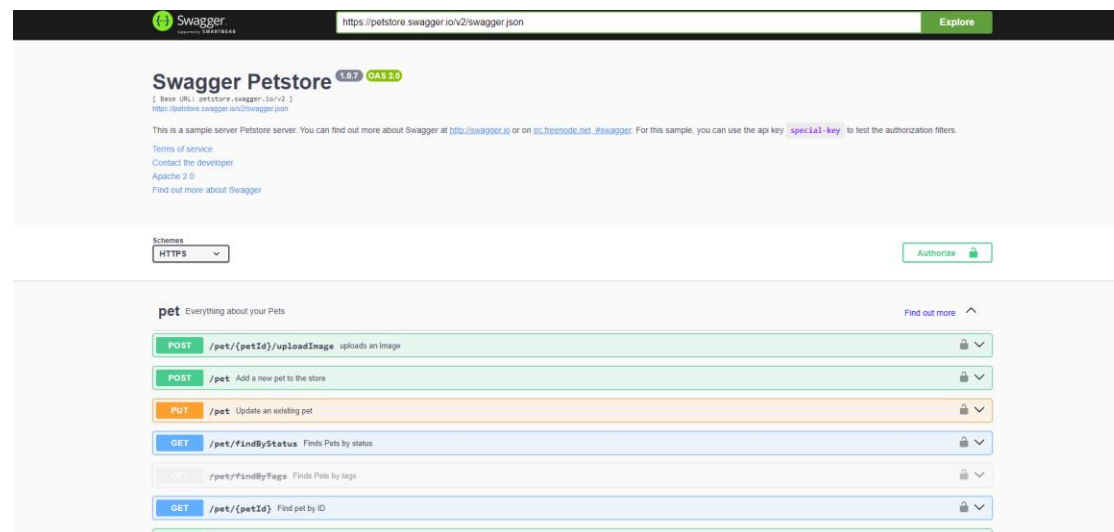
Swagger UI - Tạo tài liệu tự động

Swagger UI là gì?

Công cụ đọc file OpenAPI và tạo **documentation website**.

-  Hiển thị tất cả endpoints
-  Parameters và request/response schemas
-  Thử nghiệm API trực tiếp trong browser
-  Generate code examples

Giao diện Swagger UI



Triển khai Swagger UI với Flask và Flask-RESTX

Bước 1: Cài đặt package Flask-RESTX

```
pip install flask-restx
```

Bước 2: Định nghĩa các model

```
login_model = api.model('Login', {
    'username': fields.String(required=True, default='admin'),
    'password': fields.String(required=True, default='123456')
})

book_model = api.model('Book', {
    'id': fields.Integer(readOnly=True),
    'title': fields.String(required=True),
    'author': fields.String(default="Unknown"),
    'year': fields.Integer(default=2026)
})
```

Flask-RESTX sẽ tự động đọc cấu trúc Class và Model để sinh ra tài liệu. Tạo sự nhất quán giữa tài liệu và code.

Bước 3: Truy cập

```
http://127.0.0.1:5000/apidocs
```

Ví dụ: POST /api/v1/books

```
@ns_books.route('')
class BookList(Resource):
    @jwt_required()
    @api.expect(book_model)
    @api.response(201, 'Created')
    @api.response(400, 'Bad Request')
    def post(self):
        """Add new book"""
        data = request.json
        if not data or 'title' not in data:
            return wrap_with_metadata_error("bad-request"), 400

        new_book = {
            "id": books_db[-1]['id'] + 1 if books_db else 1,
            "title": data['title'],
            "author": data.get('author', "Unknown"),
            "year": data.get('year', 2026)
        }
        books_db.append(new_book)
        return wrap_with_metadata(new_book), 201
```

Kết quả

api/v1/books

Book Management

^

GET

/api/v1/books

Get all books

⌵

🔒

POST

/api/v1/books

Add new book

^

🔒

Parameters

Try it out

Name

Description

payload

* required

object

Example Value | Model

(body)

```
{
  "id": 0,
  "title": "string",
  "author": "Unknown",
  "year": 2026
}
```

Parameter content type

application/json

⌵

Responses

Response content type

application/json

⌵

Code

Description

201

Created

400

Bad Request

GET

/api/v1/books/{book_id}

Get a specific book

⌵

🔒